

**Unit – II Introduction to Embedded Systems**

**Introduction to Embedded Systems**

An **embedded system** is a specialized computing system designed to perform a specific set of tasks or functions within a larger system. Unlike general-purpose computers (such as desktops, laptops, or servers), embedded systems are typically designed to work with specific hardware and software to accomplish predefined objectives, often with strict real-time constraints.

These systems are "embedded" within larger devices or products, ranging from household appliances and automotive systems to industrial machinery and medical equipment. The key characteristic of embedded systems is that they are application-specific, meaning they are built to serve a specific function rather than to be general-purpose computing platforms.

**Types of Embedded Systems**

**1. Standalone Embedded Systems:**

- These systems operate independently and do not require a host system. Examples include digital watches, microwave ovens, and vending machines.

**2. Real-Time Embedded Systems:**

- These systems must meet specific timing constraints and provide **real-time responses** to inputs. Examples include automotive systems like airbags, industrial controllers, and medical devices.

**3. Networked Embedded Systems:**

- These systems are connected to other systems or devices over a network. They may communicate with a central server or other embedded systems, often via **Wi-Fi, Bluetooth, or Ethernet**. Examples include **smart thermostats** or **IoT (Internet of Things) devices**.

**4. Mobile Embedded Systems:**

- Embedded systems integrated into portable, mobile devices such as **smartphones, wearables, and portable medical devices**. These systems require efficient power consumption and high performance in a small form factor.

**Architecture of Embedded Systems ( components of embedded system ):**

The architecture of an embedded system refers to the structure and organization of the system's hardware and software components that work together to perform a specific task. The architecture defines how the system operates and how it interfaces with the external world, including sensors, actuators, and other devices.

An embedded system architecture typically involves the following main components:

**A . Hardware Architecture :**

The hardware of an embedded system consists of several key components that are responsible for executing the software and interacting with external devices. These components include:

## 1. Microcontroller / Microprocessor (CPU):

- **Microcontroller (MCU):** It is the heart of an embedded system. It contains a **processor**, **memory**, and **I/O peripherals** on a single chip. Microcontrollers are used in small, low-power applications like home appliances, toys, and portable gadgets.
- **Microprocessor (MPU):** Microprocessors are typically used in more complex systems with higher processing power. They usually require external components like RAM, ROM, and I/O devices. Microprocessors are commonly used in devices like laptops, smartphones, and embedded computers.

Examples:

- **8051** (Microcontroller)
- **ARM Cortex-M** (Microcontroller), Advanced RISC Machine
- **Intel 80386** (Microprocessor)

## 2. Memory:

- Embedded systems typically use two types of memory:
  - **ROM (Read-Only Memory):** Used to store the firmware (permanent software). It retains its content even when the system is powered off.
  - **RAM (Random Access Memory):** Temporary memory used for data storage during system operation. RAM is volatile, meaning it loses data when power is lost.
  - **Flash Memory:** Non-volatile memory that is often used for both firmware storage and data storage in embedded systems.

Types of memory commonly used in embedded systems:

- **EEPROM (Electrically Erasable Programmable Read-Only Memory):** Stores small amounts of data that can be modified electrically.
- **SRAM (Static RAM):** A faster but more expensive memory type used for short-term data storage.

## 3. Input/Output (I/O) Devices:

- Embedded systems interact with the external environment through various I/O devices. The I/O devices can be either **digital** or **analog**.
  - **Digital I/O:** Includes components like switches, buttons, LEDs, and displays.
  - **Analog I/O:** Includes sensors and actuators that provide continuous signals, such as temperature sensors, light sensors, motors, etc.

Example of I/O peripherals:

- **GPIO (General Purpose Input/Output)** pins for controlling LEDs, buttons, etc.
- **ADC (Analog-to-Digital Converter)** to interface analog sensors.
- **DAC (Digital-to-Analog Converter)** to control analog actuators.

- **Sensor** : Detects changes in environment ( temperature , light , motion ), and provides input.
- **Actuator** : Performs action based on the system output , example – motor , relay .

#### 4. **Communication Interfaces:**

- Communication modules are used to connect the embedded system with other devices or networks.
- Common communication protocols used in embedded systems include:
  - **UART (Universal Asynchronous Receiver/Transmitter)**: For serial communication.
  - **SPI (Serial Peripheral Interface)**: For high-speed data transfer.
  - **I2C (Inter-Integrated Circuit)**: For communication with sensors and other devices.
  - **CAN (Controller Area Network)**: Used in automotive systems for communication between electronic control units (ECUs).
  - **Ethernet, Wi-Fi, Bluetooth** for network communication.

#### 5. **Power Supply:**

- Embedded systems are often battery-powered or have a dedicated power source (e.g., **DC adapters, solar power, battery packs**).
- Low power consumption is a key consideration in embedded system design to maximize battery life and reduce energy costs.

## **B . Software Architecture :**

The software architecture of an embedded system is responsible for managing the system's hardware and executing the logic that drives the system's functionality. The software typically interacts directly with hardware and uses various software components, such as drivers, applications, and middleware.

#### 1. **Firmware:**

- **Firmware** is the permanent software embedded into the system. It is often stored in **ROM** or **Flash memory**. The firmware controls the hardware and implements low-level functionality, such as device initialization and communication with sensors or actuators.

#### 2. **Embedded Operating System (RTOS):**

- Some embedded systems use an **Operating System** to manage tasks, scheduling, and resource allocation. In **real-time embedded systems**, the operating system used is often a **Real-Time Operating System (RTOS)**, which ensures that tasks are executed within a specified time frame.
- Examples of RTOS include:
  - **FreeRTOS**: A lightweight RTOS used for small embedded systems.
  - **uC/OS-II**: An RTOS suitable for real-time applications.
  - **RTEMS**: Real-Time Executive for Multiprocessor Systems.
  - **Linux**: For more complex embedded systems with higher processing power (e.g., embedded Linux on Raspberry Pi).

However, many small embedded systems do not require an OS and operate **bare-metal** (without an OS), where the software directly manages the hardware.

### 3. Device Drivers:

- **Device drivers** are specialized software components that control and manage communication between the hardware and software. Drivers allow the system to interface with peripherals like sensors, displays, and motors.
- For example, a driver for a **temperature sensor** would read data from the sensor and provide it to the software for processing.

### 4. Middleware:

- **Middleware** refers to the software that provides services and functions for communication between applications and hardware. Middleware can also include communication protocols or libraries for network connectivity.
- Examples include **TCP/IP stacks**, **Bluetooth libraries**, and **file systems** for embedded systems.

### 5. Application Layer:

- The **application layer** consists of the high-level software responsible for the functionality of the embedded system. It processes data from sensors, controls actuators, and provides user interfaces such as displays or alarms.
- In more complex embedded systems, the application layer may also include user interface elements, such as touchscreens or graphical displays.

## C . Embedded System Architecture Models :

Embedded systems can be categorized into different architectural models based on their complexity and purpose:

### 1. Single-Core Architecture:

- A **single-core** embedded system architecture has one central processing unit (CPU) that handles all tasks and processes. This is common in simpler embedded systems such as microcontrollers.
- Examples: **8051**, **PIC ( Peripheral Interface Controller )**, **AVR ( Advanced Virtual RISC )**, **ARM Cortex-M**.

### 2. Multi-Core Architecture:

- Multi-core embedded systems have more than one CPU or processor core, allowing parallel processing and multitasking. This is useful for more complex tasks or applications that require higher performance, such as multimedia processing or networking.
- Examples: **ARM Cortex-A** series, **Raspberry Pi** , **SBC ( single board computer )**

### 3. Distributed Architecture:

- In **distributed systems**, multiple embedded processors work together to handle different tasks or components of the system. Each processor is dedicated to a specific function, such as sensor data collection, processing, and communication.

- Example: **Automotive control systems** often use distributed microcontrollers for different subsystems (e.g., engine control, braking, infotainment).

## **D. Embedded System Design Flow ( Development Process ) :**

The design of an embedded system typically involves several steps:

### **1. Requirement Analysis:**

- Identify the system's functional and non-functional requirements, such as performance, power consumption, cost, and timing constraints.

### **2. System Design:**

- Select appropriate components (e.g., microcontroller, memory, sensors, and communication interfaces) and design the system architecture, considering hardware and software requirements.

### **3. Software Development:**

- Develop the embedded software to control the hardware and implement the required functionality, either using bare-metal programming or by using an RTOS.

### **4. Prototyping:**

- Create a prototype to test the design, which may involve assembling the hardware and integrating it with the software.

### **5. Testing and Debugging:**

- Test the embedded system to ensure it functions correctly under various conditions. Debug any issues related to hardware, software, or communication.

### **6. Deployment and Maintenance:**

- Deploy the system in the real-world environment, and periodically update or maintain the system, such as firmware updates or performance enhancements.

## **Characteristics of Embedded Systems :**

### **1. Dedicated Functionality:**

- Embedded systems are built to perform a **specific function** or task. For example, a washing machine controller, a digital camera, or a microwave oven all have dedicated functionalities they are designed to handle.

### **2. Real-Time Operation:**

- Many embedded systems are **real-time systems**, meaning they must respond to inputs or events within a guaranteed time frame. For instance, in automotive systems like airbags, the response time must be fast and predictable.

### **3. Resource Constraints:**

- Embedded systems often operate in environments where resources such as **memory**, **processing power**, and **battery life** are limited. This requires careful consideration in design to optimize code and hardware usage.

### **4. Interaction with Hardware:**

- Embedded systems interact closely with **hardware components** such as sensors, actuators, and motors. They may also use **peripherals** like displays, keypads, and communication interfaces.

#### 5. **Low Power Consumption:**

- Since many embedded systems are designed to operate in environments where power consumption is critical (e.g., battery-powered devices), they are typically optimized for **low power** operation.

#### 6. **Reliability and Stability:**

- Given that embedded systems are often deployed in safety-critical applications (such as medical devices or automotive systems), they must be **highly reliable** and **stable** over time.

#### 7. **Long Lifecycle:**

- Many embedded systems are designed for long-term use and may be deployed for years without needing an update or replacement, requiring **durability** and **sustainability**.

### **Applications of Embedded Systems :**

Embedded systems are ubiquitous and play a critical role in a wide range of industries and everyday products:

#### 1. **Consumer Electronics:**

- Embedded systems are used in devices like **televisions, digital cameras, microwave ovens, washing machines, smartphones, and smart TVs**.

#### 2. **Automotive Systems:**

- Embedded systems are widely used in modern vehicles for controlling functions like **engine management, braking systems, airbags, infotainment, and navigation**.

#### 3. **Industrial Automation:**

- **Programmable Logic Controllers (PLCs) and SCADA( Supervisory Control And Data Acquisition ) systems** are used for controlling manufacturing processes, robotics, and automation in factories.

#### 4. **Medical Devices:**

- Embedded systems are integral to devices like **pacemakers, insulin pumps, patient monitors, and diagnostic equipment**.

#### 5. **Home Automation and IoT:**

- **Smart home devices like thermostats, security systems, light control systems, and voice assistants** all rely on embedded systems.

#### 6. **Aerospace and Defense:**

- Embedded systems are used in **navigation systems, drones, radar systems, and military equipment**.

#### 7. **Telecommunications:**

- Embedded systems are used in networking equipment like **routers**, **switches**, and **modems** for **data transmission**, **signal processing**, and **network management**.

## **Embedded C and Development Tools :**

Embedded C refers to the use of the C programming language to develop software applications for embedded systems. Embedded systems are specialized computing systems that are designed to perform specific tasks and are often constrained in terms of processing power, memory, and input/output (I/O) capabilities. Examples of embedded systems include microcontrollers in appliances, automotive control systems, and wearable devices.

Embedded C is an extension of the standard C language tailored to meet the specific requirements of embedded systems development. It provides features for working with hardware, low-level control, and real-time constraints.

### **A. Embedded C Overview**

Embedded C is similar to traditional C but includes features and libraries designed for interacting directly with hardware and low-level resources. It supports:

- **Direct hardware access:** Accessing and manipulating microcontroller registers and peripherals (GPIO, timers, ADC, UART, etc.).
- **Memory management:** Handling memory constraints through pointers and fixed-size buffers.
- **Interrupt handling:** Writing interrupt service routines (ISR) to handle asynchronous events.
- **Real-time constraints:** Writing time-sensitive applications that must meet strict timing requirements.

### **Features of Embedded C :**

#### **1. Portability:**

- Embedded C code is portable across different microcontroller platforms, with minimal modifications required. However, it may still need hardware-specific code for peripherals.

#### **2. Low-Level Programming:**

- Embedded C is designed to interact with the hardware at a low level, including manipulating registers, performing bitwise operations, and working with timers, interrupts, and sensors.

#### **3. Efficiency:**

- Embedded systems are often resource-constrained, so Embedded C code is optimized for speed and memory usage. This is critical for running on devices with limited processing power and memory.

#### **4. Real-time Support:**

- Embedded C supports real-time programming, allowing developers to create systems that meet specific timing requirements or constraints (e.g., real-time clock, real-time scheduling).

#### **5. Interrupt Handling:**

- The language supports defining interrupt service routines (ISRs), allowing the system to respond to hardware events or signals asynchronously.

## B. Embedded C Development Tools

To develop embedded systems with Embedded C, several development tools are required to compile, debug, and flash the code onto microcontrollers. Below are the key tools used in the embedded development process:

### 1. Integrated Development Environment (IDE)

An IDE provides a comprehensive platform for writing, compiling, debugging, and testing embedded C applications. Some of the popular IDEs for Embedded C development are:

- **Keil  $\mu$ Vision:**

Keil refers to surname, it is a German word

- Keil  $\mu$ Vision is one of the most widely used IDE (Integrated Development Environment) for embedded development, particularly for 8051 and ARM Cortex microcontrollers.
- It supports features like real-time debugging, code optimization, and simulation of peripherals.
- Keil provides powerful debugging tools like breakpoints, watchpoints, and step-through execution.

- **IAR Embedded Workbench:**

IAR (Ingenjorsfirma Anders Rundgren) is a company that develops C, C++, compilers

- IAR Embedded Workbench is another professional IDE used for developing embedded systems applications.
- It provides a wide array of debugging and optimization tools and supports various microcontrollers, including ARM, 8051, and AVR.

- **Atmel Studio:**

Atmel (Advanced Technology for memory and logic)

- Atmel Studio is an IDE for AVR and ARM-based microcontrollers, specifically for Atmel (now part of Microchip) products.
- It integrates well with AVR and ARM toolchains and includes many debugging features for embedded applications.

- **Eclipse + GCC (GNU Compiler Collection):**

- Eclipse is an open-source IDE, and with the GCC toolchain, it can be used for embedded development.
- It's commonly used with ARM Cortex-M or other microcontrollers and allows customization and flexibility for large-scale projects.

- **MPLAB X IDE:**

MPLAB (Microchip PIC macro assembler)

- Developed by Microchip, MPLAB X IDE is used for programming PIC and dsPIC microcontrollers.
- It provides debugging, simulation, and hardware programming support.



## 2. Compilers

The compiler is responsible for converting the C code into machine code that the microcontroller can execute. Some common compilers for embedded systems are:

- **Keil C Compiler:**
  - The Keil C compiler is a widely used compiler for 8051 and ARM Cortex-M microcontrollers.
  - It generates highly optimized code for the target microcontroller.
- **GCC (GNU Compiler Collection):**
  - GCC is an open-source compiler that supports a variety of architectures, including ARM, AVR, and x86.
  - It is highly portable and used with several IDEs like Eclipse or Atmel Studio.
- **IAR Embedded Workbench Compiler:**
  - IAR provides a proprietary compiler for embedded development with optimizations specifically designed for embedded systems.
- **SDCC (Small Device C Compiler):**
  - SDCC is an open-source compiler for 8-bit and 16-bit microcontrollers (e.g., 8051, Z80, PIC).
  - It is lightweight and commonly used in educational and hobbyist projects.

## 3. Debuggers

A debugger allows you to inspect the internal state of the embedded system during execution, set breakpoints, view variables, **and step through code.**

- **JTAG/SWD Debuggers:**
  - JTAG (Joint Test Action Group) and SWD (Serial Wire Debug) are two popular debugging interfaces used to interface with microcontrollers.
  - Segger J-Link, ST-Link, and CMSIS (common microcontroller software interface standard)-DAP are examples of hardware debuggers that allow real-time debugging over JTAG or SWD.
- **On-Chip Debugging (OCD):**
  - Many microcontrollers provide on-chip debugging support, where the debugger interfaces directly with the microcontroller to control the execution and monitor hardware.
- **Simulator:**
  - Many IDEs such as Keil and IAR provide software simulators that allow you to simulate the execution of the program on a virtual microcontroller without requiring physical hardware.

## 4. Flashing Tools and Programmers

Once the program is compiled and debugged, it needs to be loaded into the microcontroller. Flashing tools are used for this purpose.

- **USB Programmers/Flashing Devices:**

- USB to serial converters (like USBasp, ST-Link, or J-Link) are commonly used to flash programs to the microcontroller.
- Flash Magic (for 8051 microcontrollers) is a software tool used to upload the firmware over serial communication.

- **In-System Programmers (ISP):**

- ISP allows programming microcontrollers in-circuit (without removing them from the hardware setup).
- Tools like AVRISP, Pickit, or J-Link allow developers to program microcontrollers in the field.

## 5. Emulators

An emulator is used to simulate the microcontroller's hardware and run code without requiring the actual hardware. Emulators allow for debugging and testing on the host computer before flashing the actual hardware.

- **Proteus:**

- Proteus is a simulation software widely used for simulating embedded systems. It supports microcontrollers like 8051, AVR, PIC, and ARM and can simulate the behavior of both hardware and software.

- **QEMU (Quick Emulator):**

- QEMU is an open-source emulator that supports ARM, x86, and other architectures. It is useful for running embedded software on the host computer and debugging before using actual hardware.

## 6. Real-Time Operating Systems (RTOS)

An RTOS is used when developing complex embedded systems that require real-time capabilities. It provides mechanisms for multitasking, scheduling, and synchronization.

- **FreeRTOS:**

- FreeRTOS is a widely used open-source RTOS for embedded systems. It supports various microcontroller architectures and provides task scheduling, inter-task communication, and resource management.

- **uC/OS-II or uC/OS-III:**

- uC/OS is a real-time operating system designed for small embedded systems. It provides basic multitasking features suitable for microcontrollers with limited resources.

- **CMSIS RTOS:**

- CMSIS RTOS is a lightweight RTOS that is often used in ARM Cortex-M systems. It is part of the CMSIS (Cortex Microcontroller Software Interface Standard) from ARM.

## Introduction to Keil $\mu$ Vision

**Keil  $\mu$ Vision** is a powerful **integrated development environment (IDE)** and **compiler** used for programming and developing applications for **microcontrollers (MCUs)**. It is widely used for **embedded**

**systems development**, especially for microcontrollers from various families such as **8051**, **ARM**, **Cortex-M**, **AVR**, **STM32**, and more.

Keil  $\mu$ Vision is known for its user-friendly interface, ease of use, and integration with the **ARM** development ecosystem. It supports the entire development process, including coding, compiling, debugging, and deploying embedded applications to target hardware.

### Key Features of Keil $\mu$ Vision

1. **Code Editor:** The IDE provides a sophisticated code editor with syntax highlighting, code completion, and debugging features that help in writing embedded C, C++, or assembly language programs.
2. **Compiler:** Keil  $\mu$ Vision includes **C51** (for 8051), **ARM Compiler** (for ARM-based microcontrollers), and other specific compilers. It allows for easy translation of high-level languages (like C) into machine code that can be executed on microcontrollers.
3. **Debugger:** Keil  $\mu$ Vision comes with a built-in debugger that helps test and debug the program on both **hardware** and **simulation**. It allows users to monitor registers, memory, and peripherals, and step through the code to analyze program flow.
4. **Simulation:** It allows you to simulate your embedded application in the absence of hardware. This feature is crucial for **hardware-in-the-loop** testing and software validation before deployment.
5. **Project Management:** Keil  $\mu$ Vision supports project management, including easy-to-use **linker**, **compiler options**, and **device configuration**. You can easily add and organize files, define target hardware configurations, and adjust project settings.
6. **Real-Time Operating System (RTOS) Support:** Keil  $\mu$ Vision provides support for **RTOS** (Real-Time Operating Systems) like **RTX** for ARM and **Keil RTX** for embedded systems. This helps in managing tasks and threads within an embedded system.
7. **Target Device Support:** Keil  $\mu$ Vision supports a wide range of microcontrollers from different manufacturers like **Atmel**, **STMicroelectronics**, **NXP**, **Texas Instruments**, **Infineon**, and **Silicon Labs**, and offers built-in support for various MCUs.
8. **Peripheral Simulation:** The IDE allows simulation of many peripheral devices, like **GPIO**, **USART**, **ADC**, **Timers**, and **PWM**, which enables testing of complex embedded applications.
9. **Code Optimization:** Keil's compiler offers a wide range of optimization options that are designed to reduce the code size and increase the speed of execution. This is particularly useful for embedded systems with limited resources.

### Components of Keil $\mu$ Vision

Keil  $\mu$ Vision consists of the following key components:

1.  **$\mu$ Vision IDE (Integrated Development Environment):** This is the main interface that combines the editor, debugger, and project manager into a unified interface. It is used for writing, compiling, and debugging programs.
2.  **$\mu$ Vision Compiler:** The  $\mu$ Vision compiler (like **C51** or **ARM Compiler**) converts the high-level C code into machine code that the microcontroller can execute. Keil provides specific compilers for different microcontroller families.

3. **Debugger:** The debugger helps you to **step through your code**, **monitor variable values**, and **simulate real-world behavior** to ensure the system operates as expected.
4. **Device Database:** Keil maintains a device database that contains detailed information about the microcontroller hardware. This allows you to select the specific microcontroller you're working with, configure it in the IDE, and generate the correct machine code.
5. **Simulation Tools:** Keil provides simulation tools for simulating various peripherals (like UART, GPIO, ADCs, etc.) so that you can test your code even before you have access to the hardware.

## Basic Workflow in Keil $\mu$ Vision

The general steps for using Keil  $\mu$ Vision in embedded system development are:

1. **Create a New Project:**
  - Choose your target microcontroller or processor.
  - Select the appropriate **toolchain/compilers** (e.g., C51 for 8051, ARM for ARM-based microcontrollers).
2. **Write Your Code:**
  - Create new **C files** or **Assembly files** for your program logic.
  - Use the built-in editor to write and modify your code.
3. **Compile the Code:**
  - Once the code is written, click the **Build** button to compile the project. The Keil compiler converts the high-level code into machine language for the target microcontroller.
4. **Debug and Simulate:**
  - Use the **Debug** feature to test and step through your code.
  - Set breakpoints, monitor variable values, and check the behavior of the program during runtime.
  - Optionally, simulate the behavior of hardware peripherals like timers, UART, and GPIO.
5. **Download to Target Device:**
  - After debugging and testing, you can download the compiled code onto the actual microcontroller via a **programming interface** (e.g., JTAG, ISP, UART, etc.).
6. **Run the Application:**
  - Run your embedded application on the target hardware and ensure it works as expected.

## Supported Microcontrollers and Architectures

Keil  $\mu$ Vision supports a broad range of microcontroller architectures, including:

1. **8051-based Microcontrollers:**
  - Keil's **C51** compiler supports 8051 series microcontrollers. You can write programs for **8051** and **51-family** chips using this compiler.
2. **ARM Cortex-M:**
  - Keil is best known for its support of ARM-based microcontrollers. The **ARM Compiler** is used for **Cortex-M**, **Cortex-A**, and **Cortex-R** processors.

### 3. AVR:

- Keil supports **AVR** microcontrollers from **Atmel** (now part of Microchip Technology), making it possible to develop for those devices too.

### 4. Other Microcontrollers:

- Keil  $\mu$ Vision supports a variety of other manufacturers and architectures, including **PIC**, **STM32**, **NXP**, and more.

## Key Advantages of Using Keil $\mu$ Vision

1. **Wide Hardware Support:** Keil  $\mu$ Vision supports a large variety of microcontroller families, making it suitable for development across many platforms.
2. **Ease of Use:** The interface is intuitive and beginner-friendly, making it an excellent tool for new embedded system developers. The graphical representation of variables, peripherals, and settings aids in efficient development.
3. **Integrated Debugging:** The built-in debugger allows you to test your code in real-time and even simulate how it will interact with hardware, which is useful if your hardware is not yet available.
4. **Optimization:** Keil's **compiler optimizations** help reduce code size and improve runtime performance, which is crucial in resource-constrained embedded systems.
5. **Support for RTOS:** It supports real-time operating systems like **RTX**, which helps developers manage multi-threaded tasks and scheduling for complex embedded applications.
6. **Device-Specific Libraries:** Keil provides libraries and middleware for common tasks like **I2C**, **SPI**, **USB**, **Ethernet**, and other communication protocols.
7. **Cross-Platform Development:** Although Keil is primarily used on **Windows**, it is possible to develop for various target platforms, from small 8-bit microcontrollers to complex 32-bit systems.

## Programming with Keil $\mu$ Vision

Programming with **Keil  $\mu$ Vision** involves several steps, from setting up the project to writing the code, compiling it, debugging, and eventually flashing the code onto the microcontroller. Here's a step-by-step guide to get you started with programming using **Keil  $\mu$ Vision** for embedded systems development.

### 1. Install Keil $\mu$ Vision

- **Download:** First, you need to download and install Keil  $\mu$ Vision from the [Keil website](#).
- **Installation:** Follow the instructions to install the software. During the installation, ensure that you select the appropriate device family (e.g., **8051**, **ARM Cortex-M**, **STM32**) based on the microcontroller you are using.

### 2. Create a New Project

To begin coding, you need to set up a new project in Keil  $\mu$ Vision.

- **Step 1:** Open **Keil  $\mu$ Vision** IDE.
- **Step 2:** Go to **Project** → **New Project** and give your project a name (e.g., **LED\_Blink**).
- **Step 3:** Select the directory where you want the project to be saved.
- **Step 4:** You will be prompted to select the **target device** (microcontroller). Choose the correct microcontroller model or family, such as **8051** or **ARM Cortex-M**.

- For example, if you're using an **8051-based** microcontroller, select an 8051 chip like **AT89C51**.
- **Step 5:** After selecting the target device, Keil will ask if you want to add the startup code (default settings for the chosen microcontroller). Choose **Yes** to add the startup code.

### 3. Add Source Files

- **Step 1:** Right-click on **Source Group 1** under the **Project window** in the left panel.
- **Step 2:** Select **Add New Item to Group 'Source Group 1'**.
- **Step 3:** Choose the **C File** option and give it a name (e.g., main.c).
- **Step 4:** Now you can start writing your embedded program in this file.

### 4. Writing Code in C

Keil µVision supports programming in **C**, **C++**, and **Assembly**. For embedded systems, C is the most commonly used language due to its simplicity and efficiency.

Here is an example of a simple **LED blinking program** for an 8051 microcontroller:

#### Example: LED Blink Program (8051)

```
c
Copy
#include <reg51.h> // Include 8051-specific header file

#define LED P1_0 // Define the LED pin (P1.0)

void delay(unsigned int time) {
    unsigned int i, j;
    for(i = 0; i < time; i++) {
        for(j = 0; j < 1275; j++);
    }
}

void main() {
    while(1) {
        LED = 1; // Turn on LED (P1.0 HIGH)
        delay(500); // Delay
        LED = 0; // Turn off LED (P1.0 LOW)
        delay(500); // Delay
    }
}
```

In this example:

- The program blinks an LED connected to **P1.0** of the **8051 microcontroller**.

- The `delay()` function is used to create a time delay, making the LED blink at regular intervals.
- 

## 5. Compile the Code

- **Step 1:** Click on the **Project** menu and select **Build Target** or simply press **F7** to compile the code.
- **Step 2:** Keil will check your code for errors. If there are any errors, they will be shown in the **Build Output** window.
  - If the code is error-free, it will generate an **.hex** file (the executable code for the microcontroller) in the **Output** folder of your project directory.

You will see the following:

- **Errors:** If any syntax or logic issues exist in the code.
  - **Warnings:** These indicate possible issues but are not critical for compilation.
  - **Hex file generated:** This is the file to load onto the microcontroller.
- 

## 6. Debugging the Code (Optional)

Keil  $\mu$ Vision provides a **debugger** that allows you to step through your code, inspect registers, memory, and variables, and simulate the program's behavior without needing physical hardware.

- **Step 1:** Set breakpoints by clicking on the left margin of the code editor next to the line numbers.
  - **Step 2:** Click on the **Debug** → **Start/Stop Debug Session** (or press **Ctrl+F5**).
  - **Step 3:** The debugger will start, and you can step through your code using **Step Over** (F10) and **Step Into** (F11). You can also monitor the **values of variables, registers, and flags** during debugging.
  - **Step 4:** After testing, click on **Stop Debug Session** to end the debug session.
- 

## 7. Flash the Code to the Target Microcontroller

Once your code is compiled and debugged, you need to **load it onto your target microcontroller** (hardware).

- **Step 1:** Connect your microcontroller to the computer via a programmer/debugger (such as **USB to serial** or **JTAG**).
- **Step 2:** In Keil, go to **Flash** → **Download** or select the appropriate option for your programmer.
- **Step 3:** Ensure that the **target device** is correctly selected.
- **Step 4:** Keil will automatically upload the compiled **.hex** file to your microcontroller.

Depending on your programmer/debugger, Keil  $\mu$ Vision may also allow you to start a **debug session** directly, which will help you verify the program on the actual hardware.

## 8. Monitor the Output

After the program is flashed onto the microcontroller, you can monitor the output based on your design. For the **LED blink program**, the LED should blink on and off at regular intervals as defined by the delay function.

## Keil $\mu$ Vision Features for Embedded Development

Here are some additional features of Keil  $\mu$ Vision that are useful in embedded development:

1. **Real-time Debugging:** Supports debugging features such as watchpoints, breakpoints, single-stepping, and register inspection.
2. **Peripheral Simulation:** Simulate peripherals like timers, UART, GPIO, ADC, etc., to test your application before hardware deployment.
3. **Target Device Support:** Keil  $\mu$ Vision supports a vast array of microcontrollers, including **8051**, **ARM Cortex-M**, **AVR**, **STM32**, and more.
4. **Cross-Platform Development:** Although Keil  $\mu$ Vision primarily works on **Windows**, you can develop code for a variety of embedded systems.
5. **Optimized Code Generation:** The Keil compiler optimizes the code for performance and memory usage, crucial for resource-constrained embedded devices.

### To get the most out of Keil $\mu$ Vision:

- Make sure to familiarize yourself with the various **microcontroller families** it supports.
- Take advantage of the **debugging** and **simulation tools** to test your code before moving to actual hardware.
- Utilize **RTOS** support if your project requires real-time scheduling and task management.

Keil  $\mu$ Vision is an excellent tool for both beginner and advanced developers working on embedded systems projects.

## Introduction to Flash Magic

**Flash Magic** is a **Windows-based** software tool that allows users to program microcontrollers, particularly those based on **8051** architecture and **ARM** processors, using the **serial (UART)** or **ISP (In-System Programming)** interface. It is commonly used to load firmware (in the form of a **HEX file**) into the target microcontroller's **Flash memory** directly from a PC, simplifying the process of programming embedded systems.

Flash Magic is developed and provided by **Embedded Systems Academy**, and it is a popular choice for programming microcontrollers in embedded system development, especially for systems based on **8051-family microcontrollers**.

### Key Features of Flash Magic

1. **Device Support:**
  - **8051 Microcontrollers:** Flash Magic is well-suited for programming 8051-based microcontrollers. It supports chips from a variety of manufacturers such as **NXP (formerly Philips)**, **Atmel**, **Silicon Labs**, and others.
  - **ARM Microcontrollers:** Flash Magic also supports **ARM-based microcontrollers**, especially those that support serial programming, making it versatile for different embedded systems.
  -



## 2. Flash Programming:

- Flash Magic allows users to program microcontrollers by uploading a **HEX file** (a binary representation of the machine code for the microcontroller) into the target device's **Flash memory**. This is the standard way of uploading the firmware onto a microcontroller.

## 3. In-System Programming (ISP):

- Flash Magic uses **ISP** to program microcontrollers. ISP allows programming of microcontrollers **without the need to remove them from the circuit**. It communicates with the target device via a serial connection (typically UART/RS232) and programs the microcontroller's memory.

## 4. User-Friendly Interface:

- Flash Magic features a simple and intuitive graphical user interface (GUI) that allows developers to quickly configure and upload programs to the microcontroller.
- It offers a **step-by-step wizard** to guide users through the programming process, making it accessible for both beginners and experienced developers.

## 5. Device Settings and Configuration:

- Flash Magic allows users to configure various **device settings**, such as **baud rate**, **programming options**, **target device type**, and more, to ensure compatibility with the microcontroller and the programming environment.
- It also provides **device-specific options**, including configuring memory settings, enabling or disabling security features, and handling bootloader configurations.

## 6. Verification and Erasing:

- After programming the microcontroller, Flash Magic can verify that the program has been correctly written into the device's memory.
- Flash Magic supports **erasing** the memory of the microcontroller, which can be necessary before reprogramming the device.

## 7. Support for Bootloaders:

- Many microcontrollers come with an integrated **bootloader** that allows them to be programmed via serial interfaces (like UART) without requiring special hardware.
- Flash Magic supports **bootloader-based programming**, which simplifies the setup by using the built-in bootloader of microcontrollers.

## 8. Error Handling and Logging:

- Flash Magic provides detailed **error messages** if there are issues during the programming process, helping users identify and resolve problems.
- It logs the entire programming process, including successful programming, verification, or any issues encountered, which aids in debugging and development.

## 9. Multiple Programming Modes:

- Flash Magic supports several programming modes such as **Full Erase**, **Partial Erase**, **Readout Protection**, and **Chip Erase**.
-

## Flash Magic Workflow

### 1. Connect Hardware:

- Connect the **target microcontroller** to the PC via a **serial (RS232)** interface, often through an **USB-to-serial converter** if the PC doesn't have a native serial port.
- Ensure the microcontroller's **bootloader** is enabled (if applicable), or the target is ready to receive programming instructions over the serial port.

### 2. Launch Flash Magic:

- Open the **Flash Magic** application on your computer.
- Select the **microcontroller device** type from the list of supported devices (e.g., **8051-family**, **ARM-based** chips).
- Set the **COM port** and **baud rate** according to your hardware setup (this should match the serial connection settings).

### 3. Load HEX File:

- Browse and select the **HEX file** you want to program onto the target microcontroller. The HEX file is the compiled version of your embedded application.

### 4. Programming Options:

- You can choose various options like **erase the flash memory**, **verify after programming**, or **set the microcontroller into bootloader mode**.
- Some devices may require special programming settings, which can be selected through the **Flash Magic interface**.

### 5. Start Programming:

- Click on **Start** or **Program** to begin the process of uploading the HEX file to the microcontroller.
- Flash Magic will use the serial interface to send the data to the microcontroller, where it will be stored in the device's flash memory.

### 6. Verify and Finish:

- After programming, Flash Magic can verify if the HEX file was correctly written to the microcontroller's flash memory. If verification passes, you can disconnect the device or reset it to start running the programmed firmware.

## Supported Microcontrollers in Flash Magic

Flash Magic supports a wide variety of microcontrollers, especially **8051 family** devices, from manufacturers such as:

- **NXP (Philips) 8051 family**
- **Atmel (Microchip) 8051 family**
- **Silicon Labs C8051 family**
- **Dallas Semiconductor 8051 family**
- **ARM-based microcontrollers** (e.g., STM32, LPC, etc.)

Each microcontroller family may have specific settings and configurations available in Flash Magic.

## **Advantages of Using Flash Magic**

### **1. Ease of Use:**

- Flash Magic is easy to set up and use, with a simple interface and clear instructions that make programming microcontrollers quick and intuitive.

### **2. No Need for a Hardware Programmer:**

- With **ISP** (In-System Programming) and the support for bootloaders, Flash Magic allows programming over the **serial interface**, meaning you don't need additional hardware programmers or tools.

### **3. Support for Multiple Devices:**

- Flash Magic supports a wide range of devices and manufacturers, making it a versatile tool for embedded system development.

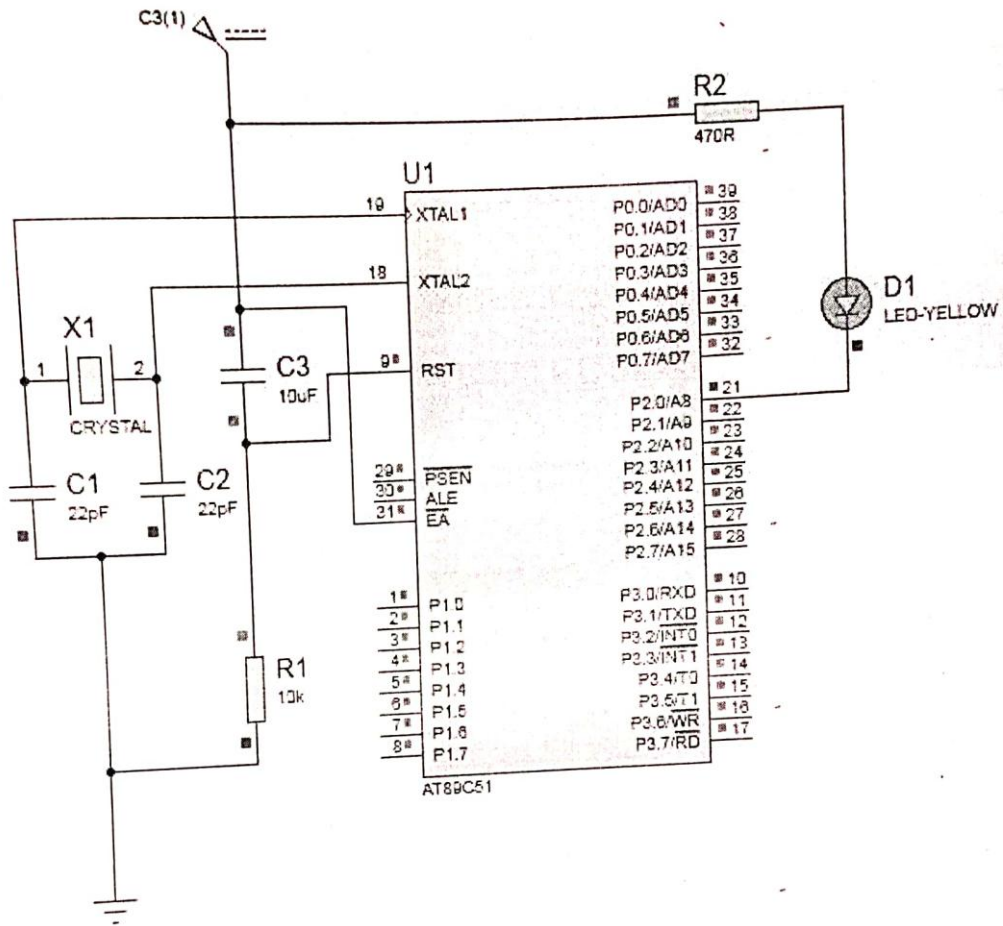
### **4. Reliable Error Checking:**

- Flash Magic includes **verification** and **error-checking** mechanisms that ensure that the microcontroller is correctly programmed, reducing the chances of failure in your embedded application.

### **5. Cost-Effective:**

- Flash Magic is typically **free for most 8051-based microcontrollers**, which makes it a cost-effective solution for programming these devices. For ARM-based systems, the software may require a license, but it is still affordable.

LED ON/OFF using Embedded C



- Now click on the **Target1** from the left panel and select **Options for Target 'Target1'**. Then set the xtal (MHz) value to 11.0592. Check mark on the Use on Chip ROM. Then go to the output tab. In this tab check **Create Hex File**, and click OK. Then build it again.

By uploading this hex file into the 8051 microcontroller the program can be loaded into it. And it will work.

#### Example

Here we have used the following code –

```
#include<reg51.h>
sbit LED_pin = P2^0; //set the LED pin as P2.0
void delay(int ms){
    unsigned int i, j;
    for(i = 0; i< ms; i++){
        // Outer for loop for given milliseconds value
        for(j = 0; j < 1275; j++){
            //execute in each milliseconds;
        }
    }
}
void main(){
    while(1){
        //infinite loop for LED blinking
        LED_pin = 0;
        delay(500); //wait for 500 milliseconds
        LED_pin = 1;
        delay(500); //wait for 500 milliseconds
    }
}
```

#### Output

The connection is like below

## Basic concepts of Designing of microcontroller based Embedded System:

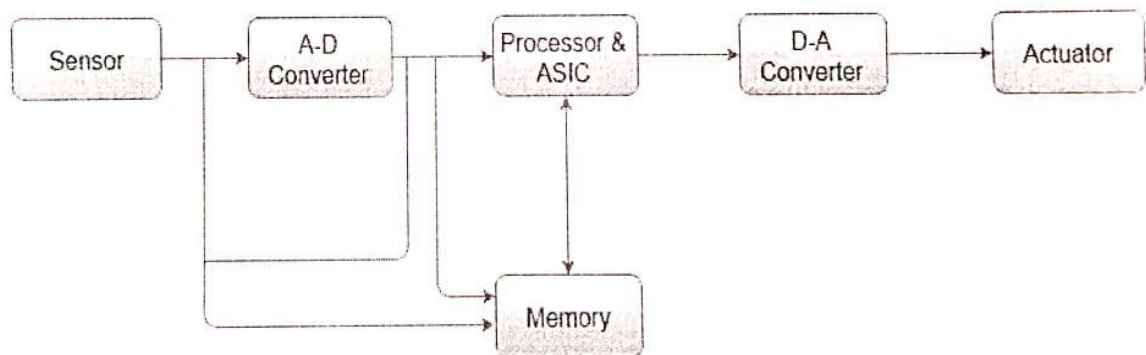
An embedded system is a microcontroller or microprocessor based system which is designed to perform a specific task. For example, a fire alarm is an embedded system; it will sense only smoke.

Embedded means something that is attached to another thing. An embedded system can be a computer hardware system having software embedded in it. An embedded system can be an independent system or it can be a part of a large system.

An embedded system has three components –

- It has hardware.
- It has application software.
- It has Real Time Operating system (RTOS) that supervises the application software and provide mechanism to let the processor run a process as per scheduling by following a plan to control the latencies. RTOS defines the way the system works. It sets the rules during the execution of application program. A small scale embedded system may not have RTOS.

So we can define an embedded system as a Microcontroller based, software driven, and reliable, real-time control system.



- **Sensor** – It measures the physical quantity and converts it to an electrical signal which can be read by an observer or by any electronic instrument like an A2D converter. A sensor stores the measured quantity to the memory.
- **A-D Converter** – An analog-to-digital converter converts the analog signal sent by the sensor into a digital signal.
- **Processor & ASICs** – Processors process the data to measure the output and store it to the memory.
- **D-A Converter** – A digital-to-analog converter converts the digital data fed by the processor to analog data
- **Actuator** – An actuator compares the output given by the D-A Converter to the actual (expected) output stored in it and stores the approved output.